

ProjectMind

Module 1 — Foundation Engine: Implementation, Changes, and Fixes

Generated: May 20, 2026

1. Introduction

ProjectMind is an autonomous, local-first AI-powered developer memory and documentation engine. It is designed to monitor AI-assisted software development projects and progressively build long-term project intelligence: detecting code changes, analyzing architecture, generating documentation, constructing Obsidian knowledge graphs, and maintaining historical development context.

Module 1 — Foundation Engine is the bedrock layer of the system. It does *not* perform any AI inference, file watching, embedding, graph generation, or git parsing. Instead, it provides the durable infrastructure on which every later module is built.

1.1 Scope of Module 1

- Foundational architecture and scalable folder structure
- Application bootstrap & graceful shutdown lifecycle
- Three-layer configuration management (YAML defaults → user config → environment overrides)
- Centralized rotating-file + colored-console logging
- Thread-safe service registry pattern for dependency-safe wiring
- Abstract base interfaces for every future module
- Obsidian-compatible markdown vault management with atomic note IO
- Shared utilities (atomic write, slugify, hashing, ISO time)
- Beginner-friendly templates for note types

1.2 Out of Scope (deferred to later modules)

- File system watchers
- AI summaries, embeddings, vector databases
- Knowledge graph generation
- Git history parsing & semantic analysis
- Autonomous agents / AI orchestration

1.3 Tech Stack

Component	Choice
Language	Python 3.12+ (runtime-compatible with 3.9)

Component	Choice
Storage	Markdown (Obsidian-compatible vault)
Config format	YAML (PyYAML)
AI runtime (future)	Ollama with Qwen models, local only
Testing	pytest
External services	None — fully local-first

2. Changes Made

The repository began as an early scaffold containing only `core/config.py`, `core/logger.py`, and a handful of empty package `__init__.py` files. Module 1 added the complete foundation engine and validated it with a 25-test suite. All changes are summarized below.

2.1 New Files Created

Core package

File	Responsibility
<code>core/registry.py</code>	Thread-safe ServiceRegistry (factory + singleton container)
<code>core/bootstrap.py</code>	Application class — startup, validation, shutdown hooks
<code>core/interfaces.py</code>	Abstract base classes for future modules
<code>core/exceptions.py</code>	ProjectMindError hierarchy
<code>core/utils.py</code>	<code>atomic_write</code> , <code>slugify</code> , <code>file_hash</code> , <code>iso_now</code> , <code>ensure_dir</code>

Obsidian vault layer

File	Responsibility
<code>obsidian/vault.py</code>	VaultManager — initialize sections, atomic read/write of notes
<code>obsidian/markdown.py</code>	Front-matter build / parse / compose helpers

Top-level project files

File	Purpose
<code>main.py</code>	Entry point with try/finally graceful shutdown
<code>requirements.txt</code>	Runtime dependency: PyYAML
<code>requirements-dev.txt</code>	Development dependency: pytest
<code>pyproject.toml</code>	Build metadata + tool config (pytest, formatters)
<code>README.md</code>	Setup, architecture, startup flow, scalability notes
<code>.gitignore</code>	Logs, vault contents, user config, <code>__pycache__</code>
<code>config/config.example.yaml</code>	User-facing example overriding defaults

Templates

- `templates/note.md` — generic note skeleton
- `templates/architecture.md` — architecture decision records
- `templates/feature.md` — feature specifications
- `templates/api.md` — API endpoint documentation
- `templates/bug.md` — bug reports
- `templates/daily.md` — daily development log

Tests

- tests/conftest.py — autouse fixtures for logger reset and env cleanup
- tests/test_config.py — defaults, deep-merge, env override, validation
- tests/test_registry.py — singleton + factory + thread safety
- tests/test_markdown.py — front-matter round-trip
- tests/test_vault.py — section initialization & atomic note IO
- tests/test_bootstrap.py — full startup & shutdown pipeline

Placeholder packages

Empty `__init__.py` stubs were replaced with descriptive docstrings (no mock implementations, per spec) in: `watcher/`, `ai/`, `analysis/`, `memory/`, `graph/`, `git/`, `intelligence/`, `docs/`.

2.2 Modified Files

File	Change
core/__init__.py	Fixed broken import; now exports ConfigLoader, ServiceRegistry, Application, ProjectMindE
obsidian/__init__.py	Exports VaultManager, build_frontmatter, parse_frontmatter, compose_note.
pyproject.toml	Tightened pytest config and tool metadata after first review pass.

2.3 Architectural Decisions

- **Service registry pattern** — every subsystem registers itself with a central `ServiceRegistry`. This avoids circular imports and makes future plugin swapping trivial.
- **Three-layer config** — defaults shipped in `config/default_config.yaml`, overridden by user `config/config.yaml` (git-ignored), then by environment variables (`PROJECTMIND_*`).
- **Atomic writes** — all note IO is written to a temp file then `os.replace()`d into place to prevent corruption on crash.
- **Abstract interfaces only** — future modules declare contracts via `core.interfaces` (e.g. `IWatcher`, `IAnalyzer`, `IMemoryStore`) without binding the foundation to a specific implementation.
- **Local-first, zero cloud** — only PyYAML is required at runtime; no Docker, no databases, no external APIs.

2.4 Validation Results

After implementation the suite was executed end-to-end:

```
$ python3 -m pytest -q
.....
25 passed in 0.4s
```

A code review pass confirmed: no broken imports, no circular dependencies, no mock implementations, full type-hint coverage, and strict adherence to the Module 1 specification (no AI, no watchers, no embeddings, no DB).

3. Errors Fixed

Each defect identified during analysis and review is recorded below alongside the resolution applied.

3.1 Broken Import in core/__init__.py

Severity: Critical (crashed on `import core`)

Symptom

```
from core.registry import ServiceRegistry
ModuleNotFoundError: No module named 'core.registry'
```

Root cause

`core/__init__.py` referenced a `ServiceRegistry` class from `core/registry.py`, but the file had never been created.

Fix

Created `core/registry.py` implementing a thread-safe service container supporting singleton instances, lazy factories, lookup by type or name, and clean teardown. Re-exported it from `core/__init__.py`.

3.2 Missing Application Bootstrap

Severity: High (no startup path existed)

Symptom

`core/logger.py` referenced `core.bootstrap` in its docstring, but no such module existed and there was no entry point.

Fix

Created `core/bootstrap.py` with an `Application` class that loads config, boots logging, ensures required directories exist, initializes the vault, registers services, and wires shutdown hooks. Created `main.py` as the user-facing entry point using `try/finally` for guaranteed cleanup.

3.3 Missing Dependency Manifest

Severity: Medium

Symptom

Code imported `yaml` but no `requirements.txt`, `pyproject.toml`, or `setup.py` declared the dependency.

Fix

Added `requirements.txt` (runtime: PyYAML), `requirements-dev.txt` (pytest), and a `pyproject.toml` with full project metadata.

3.4 Python 3.9 Runtime Incompatibility

Severity: High (tests failed at import time)

Symptom

```
TypeError: unsupported operand type(s) for |: 'type' and 'type'
```

Root cause

PEP 604 `X | Y` union syntax is only valid at runtime in Python 3.10+. The development machine ran Python 3.9, so *runtime* type aliases (e.g. `Service = type | str`) failed at module load even though `from __future__ import annotations` postponed evaluation of regular hints.

Fix

Replaced runtime PEP 604 unions with `typing.Union[...]` in `core/registry.py` (and audited the rest of the codebase). Function annotations remain modern (deferred by `__future__ import`) — the spec target of Python 3.12+ is honoured while the current dev environment runs cleanly.

3.5 Bootstrap Wrapped Domain Errors

Severity: Medium

Symptom

`Application.start()` caught all exceptions and re-wrapped them in a generic error, hiding the original `ProjectMindError` subclass and breaking `except` clauses in tests.

Fix

Bootstrap now re-raises `ProjectMindError` instances unchanged and only wraps unexpected exceptions in `BootstrapError`.

3.6 Redundant Abstract Method Re-declarations

Severity: Low (code-review finding)

Fix

Removed duplicate `@abstractmethod` declarations in `core/interfaces.py` where subclasses re-stated the same contract already inherited from a parent ABC.

3.7 Markdown Round-Trip Whitespace Drift

Severity: Low

Symptom

`compose_note(parse_frontmatter(text))` produced a string with extra blank lines vs. the original.

Fix

Normalised newline handling in `obsidian/markdown.py` so the round-trip is now byte-stable for clean inputs.

3.8 Test Pollution Across Runs

Severity: Low

Symptom

Tests sharing a global logger and reading PROJECTMIND_* env vars from previous tests produced flaky behaviour.

Fix

Added tests/conftest.py with autouse fixtures that reset the logger and clear PROJECTMIND_* env vars between tests.

3.9 Empty Placeholder Packages

Severity: Cosmetic

Fix

Replaced empty package __init__.py files with descriptive docstrings naming the future module's responsibility while remaining import-safe and adding zero behaviour (per spec).

3.10 Resolution Summary

#	Issue	Severity	Status
1	Broken ServiceRegistry import	Critical	Fixed
2	Missing bootstrap / entry point	High	Fixed
3	Missing dependency manifest	Medium	Fixed
4	Python 3.9 union-syntax runtime crash	High	Fixed
5	Bootstrap masked ProjectMindError	Medium	Fixed
6	Redundant abstract methods	Low	Fixed
7	Markdown round-trip whitespace	Low	Fixed
8	Test pollution between runs	Low	Fixed
9	Empty placeholder packages	Cosmetic	Fixed

Final state: 25 / 25 tests passing, code review clean, specification compliance verified.